

# 15.6

## Saga Pattern

Distributed transactions without 2PC

*Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples*

## The Problem: Distributed Transactions

In microservices, one business operation (like placing an order) touches multiple services. Traditional databases use ACID transactions, but you cannot do a single transaction across two different databases. The Saga pattern solves this with a sequence of local transactions and compensating actions if something fails.

■ **Simple Analogy:** Booking a holiday: you book a flight, then a hotel, then a car. If the car rental fails, you must cancel the hotel and then the flight — in reverse order. Each step is reversible. That is a Saga.

## Two Types of Sagas

| Orchestration                                                                                                                             | Choreography                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Central 'conductor' (Saga Orchestrator) tells each service what to do next. Easier to trace and debug. More coupling to the orchestrator. | Services react to events published by the previous step. No central coordinator. Loosely coupled but hard to trace. |
| <i>Best for: clear sequential flows Example: Airbnb booking</i>                                                                           | <i>Best for: simple event chains Example: order notification flow</i>                                               |

## Diagram — Orchestration Saga: Place Order

| Step | Service       | Action                 | Compensation if FAIL          |
|------|---------------|------------------------|-------------------------------|
| 1    | Order Svc     | Create order (PENDING) | Delete order                  |
| 2    | Payment Svc   | Reserve funds          | Release funds                 |
| 3    | Inventory Svc | Reserve stock          | Release stock                 |
| 4    | Shipping Svc  | Create shipment label  | Cancel label                  |
| 5    | Order Svc     | Mark order CONFIRMED   | (start of failure = undo 4→1) |

Figure 6: If step 3 fails, compensations run in reverse: release funds (2) -> delete order (1)

## Choreography Saga Example

```
// OrderService publishes event (no coordinator)
eventPublisher.publish(new OrderPlacedEvent(orderId));

// PaymentService listens and reacts
@KafkaListener(topics = "order.events")
public void on(OrderPlacedEvent event) {
    paymentService.reserveFunds(event.getOrderId());
    eventPublisher.publish(new PaymentReservedEvent(event.getOrderId()));
}

// InventoryService listens to PaymentReservedEvent next...
```

## Idempotency — Critical for Sagas

Every step in a Saga MUST be idempotent — running it twice must have the same result as running it once. Use an idempotency key (e.g., orderId) and check 'have I already processed this?' before acting. This protects against duplicate messages from Kafka.

■ **Real-World Example:** Airbnb uses Orchestration Saga for booking: Calendar block -> Charge guest -> Notify host -> Schedule payout. If any step fails, compensations run in reverse. Orchestration was chosen over choreography for full auditability of each booking step.